TUFR- Teensy Ultrasonic Field Recorder: Instructions

by Jay Barlow (jaybarlow33@yahoo.com)

Background

Field recorders are useful instruments for making long-duration recordings, such as in studies of animal sounds or anthropogenic noise. Ultrasonic recorders, with the capability to record sounds above 20 kHz, are needed for studies of bats and cetacean echolocation. There are several commercially available ultrasonic field recorders that are capable of recording a single track (mono) of ultrasound, including the low-cost AudioMoth and several bat recorders from Wildlife Acoustics. However, stereo ultrasonic field recorders have not been commercially available since Wildlife Acoustics discontinued their SM2-Bat recorder. My research on cetacean echolocation often requires stereo recordings to provide additional information on the location of the source relative to the hydrophone pair (conical bearing angle). I have been using the outstanding 4-channel recorders from Ocean Instruments (currently the Soundtrap ST-440 and ST-640), but these are very expensive (over $5000US). The lack of inexpensive, stereo, ultrasonic recorders motivated me to develop my own stereo field recording system. I did not have sufficient experience in programming micro-controllers, so I used Gemini AI as my programmer. I explored several micro-controllers and found that the Teensy 4.1 running at its minimum clock speed (24MHz) used less power during recording than the ESP32 family running at their minimum, so I based my system on that system.

Assembly

To speed development, I based this project on commercially available modules (populated circuit boards) for most functions: the Teensy 4.1 itself (without ethernet), a I2S-based 192kHz, 24-bit ADC module (Fig. 1, commercially available from many sources), a CR-battery holder (for the real-time-clock), and a Pololu 2810 digital switch. I soldered these on two generic bread boards (using female connectors in some cases) and used wire traces to connect them (Fig. 1). The wiring hookup guide is in the README.md document for this project. This project is within the capabilities of most electronic do-it-yourselfers, but is not yet available as an off-the-shelf product.

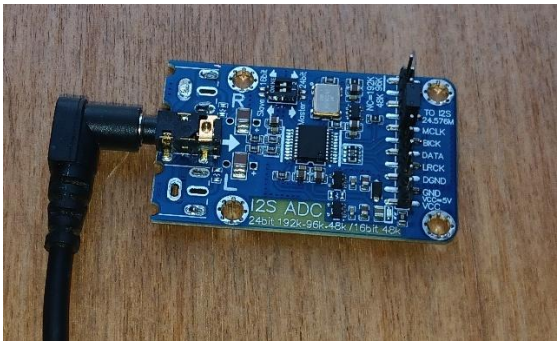

Figure 1. ADC module with I2S output based on the WM-8782 chip. DIP switches should be set to “Master” and “24-bit”. The shunt should connect the 24.576MHz oscillator pin to the MCLK pin.

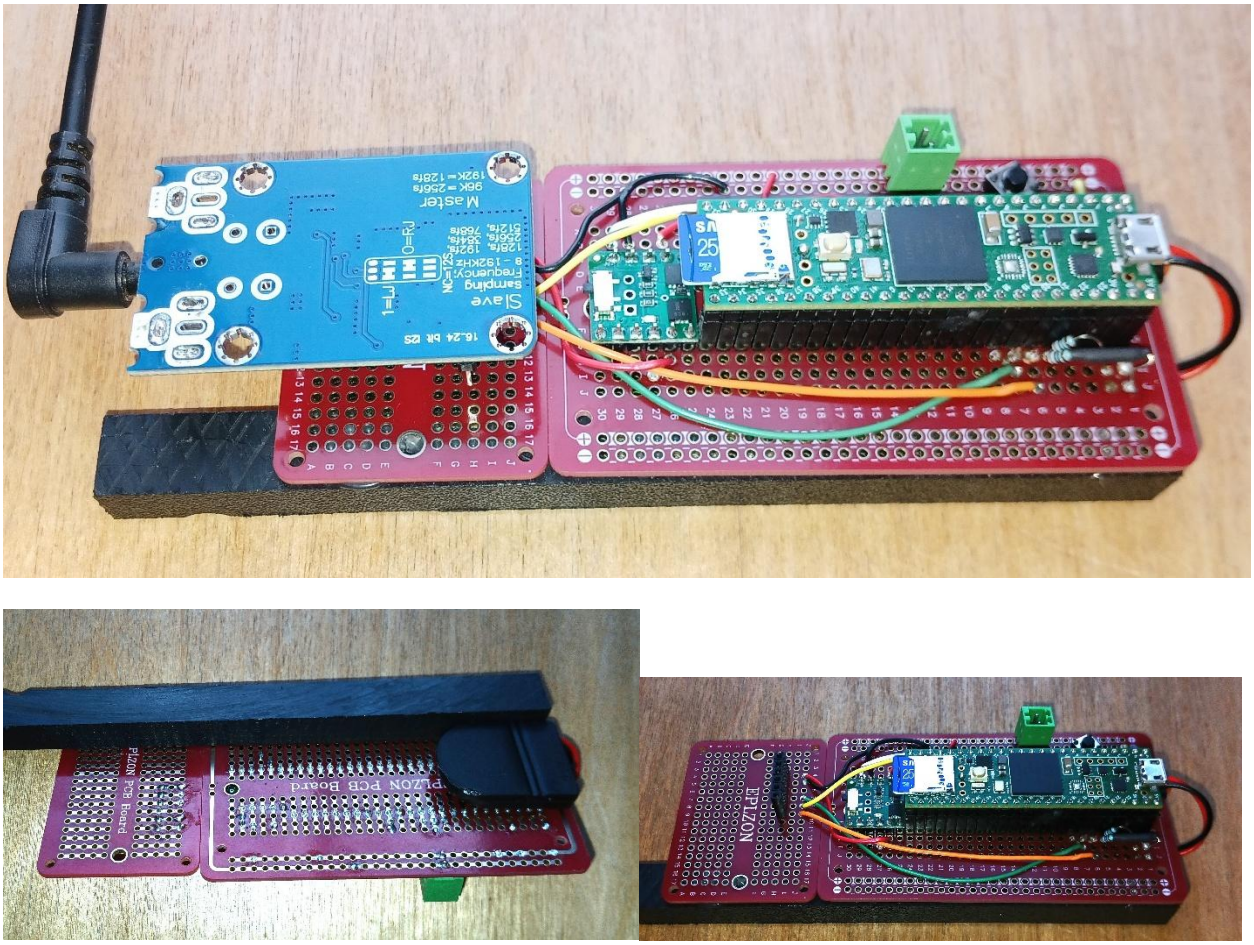

Figure 2. Assembled components, including the Teensy 4.1, Pololu 2810, I2S-ADC modules (top) and CR-battery holder (bottom left). EPLZON breadboards (red, one small and one large) are used to attach modules and are held together with black plastic bar. Female headers are used to connect the Teensy and ADC modules. The ADC module is powered by 3.3V output from the Teensy via the Pololu digital switch.

Although the recorder code is written for a 192kHz audio sampling rate, this rate can be changed to 48 or 96 kHz by changing one line of code and by adding one jumper between pins on the ADC module. This will save SD card space and a little bit of battery life.

The Teensy 4.1 microprocessor will need to be programmed using the Arduino IDE. If you haven't used this before, I recommend you watch a tutorial; many aspects are not intuitive. The code (an *.ino text file) is provided in the Code folder of this GitHub site (https://github.com/JayB545/Teensy-Ultrasonic-Stereo-Recorder).

Power Considerations

The TUFR recorder can be powered by a low-noise 5V power source either using the 5V and ground pins (recommended) or the USB connector on the Teensy board. The board requires approximately 100mA at

5V (0.5 watts) during record mode and about one third that while waiting to start recording. Actual power requirements vary with the brand and size of the micro-SD card used. Larger cards typically use more power. I have found that Samsung cards use less power than SanDisk.

Power sources less than 4V are likely to result in unstable operation and sources greater than 6V will damage the board. Most battery sources need to be conditioned to fall within this range using a boost or buck DC-DC converter. These converters use rapid switches in converting the voltage and the switching frequency introduces ripple noise. For low-noise recording and stable operation, a power source with low ripple is recommended. Choose a converter with a switching frequency that is much higher than the frequencies of interest in your recordings. Another consideration is power conversion efficiency for long battery life; buck converters (which decrease voltage) are typically more efficient at converting power (watts) than boost converters (which increase voltage). Efficiency can be up to 95% in buck converters working at their optimum input voltage. The cheapest DC-DC power supply is not likely to be the quietist or the most efficient.

Although it is tempting to power the TUFR with a USB Power Bank, most of these shut down if the power draw is too low, which commonly happens when the TUFR is waiting to start a recording period. A better solution for short recordings (up to 4 days) is a lithium-ion 18650 UPS battery holder with a built-in voltage converter and charger (such as FORIOT A42_GXHB0004-002_US). These can output 5V via a USB connector and have a “hold” switch which prevents them from shutting down if the power output is too low. They have a low-voltage cut-off to protect the batteries from damage from low voltage. A single 18650 cell will power the recorder for approximately 12 hours in recording mode, so two 4-cell battery holders would be needed for 4 days.

For longer recordings, consider using a 12V battery with a low-noise buck converter and a voltage limiter (to prevent draining the battery below allowable levels). A 100Ah 12V battery provides approximately 1200Wh of power, which would power the TUFR for 100 days (ignoring DC-DC conversion loss). For such long deployments, SD card space is likely to be limiting unless duty cycling is used.

If AC power is available at your recording site, consider powering the TUFR with a low-noise AC-DC converter like the iFi SilentPower iPower2.

Operation

Operation of the TUFR recorder is simple. Insert an appropriate micro-SD card (formatted as either ExFAT or FAT32) and apply power. The resulting WAV files are in a standard format (16-bit PCM). If present, the program will read setup options from a TUFR_SETUP.TXT file in the root directory (see README.md for setup format). If no setup information is available, it will automatically record consecutive, gap-free, 1-minute files until the battery is exhausted or until the shut-down button is pressed. When the system shuts down due to lack of power, the last file will be corrupted and unreadable. There is some chance that the file directory may also be corrupted by an uncontrolled shutdown, so I recommend that cards be re-formatted after this happens (although I have never experienced this problem). Always press the shut-down button before removing the SD card.

The LED light on the Teensy 4.1 board will flash to indicate the status of the recorder. See the README.md file for the LED codes.

The setup file allows the user to set the duty cycle (in minutes on/off), the record mode (stereo, left or right mono), reset the real-time clock, delay start until a specified time, and set the daily start and stop times. See README.md for setup format.

Future Developments

Hardware

My next project for TUFR will be to design and have built a printed circuit board to replace the breadboard and wire traces used for this prototype. This will serve as a bridge between the Teensy 4.1 and the other needed modules.

Need to re-measure the frequency response of TUFR and document the -10dB falloff between 20-90 kHz.

Test and describe hookups for common MEMS and electret microphones and piezo-ceramic hydrophones.

Software

It would be nice to have a triggered recording mode that will limit recordings to times when sounds-of-interest are present. This is commonly done for bat recordings because there is little background noise in their frequency range. It is harder to do for sound in the audible range.

I could not get a true sleep mode to work on the Teensy 4.1 when using the audio library. It would wake up immediately. The current version reduces power consumption during “wait” states by shutting down power to the ADC module and the external microphones, but the microcontroller still consumes considerable power. A deep sleep state would be preferred if this is possible.